MALLA REDDY UNIVERSITY

INTERNSHIP REPORT

ON

LLM INTEGRATION WITH GEMINI

Submitted by
Khada Suryakiran

Under the Guidance of

Rohini Reddy

Department of Computer Science & Engineering (CyberSecurity)

Malla Reddy University, Hyderabad - 500100

Academic Year: 2023–2027

ACKNOWLEDGEMENT

I'm grateful to Gaint Clout Technologies Pvt Ltd for the invaluable industrial training experience in LLM Integration and AI Technologies.

Special thanks to Rohini Reddy for their guidance and support. The challenging tasks deepened my industry knowledge. The whole Gaint Clout Technologies Pvt Ltd team's cooperation enriched my training.

Thanks to colleagues for camaraderie and knowledge sharing. This experience equips me for a successful career in AI and Cybersecurity.

I also extend heartfelt thanks to CEO Mr. Srinivas Kolakaluri and Chairman Mr. Prasad Kottamasu for providing this incredible opportunity.

Khada Suryakiran

DECLARATION

I, hereby declare that the project report titled A PROJECT REPORT ON "JARVIS – A VOICE-ACTIVATED AI ASSISTANT" is submitted by me for the partial fulfillment of the project requirements. This project is entirely my original work and has not been submitted earlier for any degree or diploma.

Place:

Date:

Khada Suryakiran

NAME & SIGNATURE OF THE CANDIDATE

ABSTRACT

This project presents an advanced integration of Google's powerful Gemini 1.5 Flash Large Language Model (LLM) and OpenAI's GPT APIs with a custom-built Flask web application, delivering a feature-rich conversational AI assistant named **Jarvis**. The system enables users to interact through a modern, responsive web interface designed with Tailwind CSS, where textual prompts can be entered manually or spoken via microphone using speech recognition.

Once a prompt is submitted, it is processed in real-time by the Gemini LLM to produce context-aware, coherent, and insightful responses, which are dynamically displayed in a styled chat window. To enhance accessibility and user experience, the system includes **text-to-speech output**, allowing users to listen to both their queries and the AI-generated responses. Additionally, the application stores the chat history with timestamps and provides functionality to **download conversation logs in both PDF and DOCX formats**, supporting documentation and future reference.

Built on Flask's lightweight backend architecture and designed for extensibility, this project serves as a practical blueprint for developers aiming to build intelligent virtual assistants, educational bots, content generators, and customer support tools. It also emphasizes secure API usage via environment variables and deployability on platforms like Render.

Overall, this system showcases how state-of-the-art language models can be seamlessly embedded into real-world web applications to provide intelligent, interactive, and multimodal user experiences. It stands as a foundation for future enhancements such as multilingual support, persistent authentication, cloud storage, and integration with other AI models.

INDEX

Contents	**Page No.**
Chapter 1 Introduction	**1 – 3**
1.1 introduction	1 - 2
1.2 Literature Servey	2
1.3 Existing System	2 - 3
Chapter 2	**4 – 6**
2.1 Proposed System	**4 - 5**
2.2 Flow Diagram	5
2.3 Working	6
Chapter 3 Implementation	**7 - 8**
Chapter 4 Source Code	**9 – 19**
4.1 Forntend	**9 - 16**
4.2 Backend	**16 - 19**
Chapter 5 Advantages and Disadvantages	**20 - 21**
Chapter 6 Results	**22 – 23**
Chapter 7 Conclusion	**24**
Chapter 8 References	**25**

List of Figures

Fig. No.	Figure Name	Page No.
Fig 2.1	System Architecture Flow	5
Fig 6.1	Home Page	22
Fig 6.2	Working of Model	22
Fig 6.3	Download of PDF	23
Fig 6.4	Download of DOCX	23

CHAPTER 1
INTRODUCTION

Large Language Models (LLMs) such as Google's **Gemini 1.5 Flash** represent a transformative leap in the fields of natural language understanding and generation. Trained on vast datasets, these models possess the capability to interpret, analyze, and produce human-like text with impressive fluency and contextual relevance. By embedding such advanced AI systems into web applications, developers can craft intelligent, responsive, and interactive user experiences that extend far beyond traditional software interfaces. This innovation paves the way for conversational agents, dynamic content creators, smart tutoring systems, and various AI-driven tools.

This project focuses on the integration of **Gemini 1.5 Flash**, one of the most powerful and efficient variants in the Gemini family, within a lightweight, modular **Flask** backend. Flask serves as the bridge between user interactions and the AI model, efficiently managing API calls, user inputs, and response rendering. The frontend, built with **HTML, JavaScript, and Tailwind CSS**, ensures a clean, responsive, and modern interface that maximizes usability and aesthetics.

Users interact with the system through a web-based interface named **Jarvis**, where they can enter prompts either by typing or using voice input through **speech recognition**. These prompts are sent to the Gemini LLM, which generates context-aware responses in real time. The results are dynamically rendered within a chat-style window, complete with user avatars, timestamps, and **text-to-speech** playback support for both user and AI responses.

Beyond interaction, the system includes features such as **PDF and DOCX download of chat history**, allowing users to export their conversations for record-keeping or educational purposes. This application not only demonstrates the seamless integration of generative AI into real-world tools but also lays the groundwork for future improvements, including authentication, persistent chat storage, multilingual interaction, and deployment on scalable cloud platforms like Render.

In essence, this project stands as a powerful example of how LLMs can be practically and effectively employed to build smart, interactive, and user-friendly web applications, opening the door to next-generation AI-powered digital experiences.

1.2 LITERATURE SURVEY

The field of conversational AI has seen rapid advancements in recent years, with the development of Large Language Models (LLMs) significantly enhancing the capabilities of AI-driven assistants. Below is a summary of existing literature and technologies that have contributed to the foundation of this project:

1. Large Language Models (LLMs)

Research by Google on the Gemini family of LLMs highlights their ability to process natural language with high fluency, accuracy, and context-awareness. Gemini 1.5 Flash, in particular, demonstrates low latency and efficiency, making it suitable for real-time applications. [Google AI Blog, 2024]

2. OpenAI's GPT APIs

The GPT-3.5 and GPT-4 models by OpenAI have been instrumental in pushing the boundaries of natural language understanding and generation. These APIs provide developers with pre-trained models capable of handling a wide range of queries, from creative writing to coding assistance. [OpenAI Documentation, 2024]

3. Voice Interaction Systems

The use of **Google SpeechRecognition API** and **gTTS (Google Text-to-Speech)** enables multimodal interaction between humans and machines. Previous work in voice interfaces has proven valuable in creating accessible and hands-free user experiences, especially in smart assistants like Alexa, Siri, and Google Assistant.

4. Web-Based Conversational Systems

Studies and implementations of chatbots using Flask and JavaScript have established the feasibility of lightweight yet powerful web-based interfaces. These systems often include features such as live chat, history tracking, and file export — components that are adopted and extended in this project.

5. AI in Assistive and Productivity Tools

Several research initiatives have explored the use of AI to build productivity boosters, educational tools, and accessibility solutions. This includes integrating LLMs with frontends that support rich UI elements, responsiveness, and voice capabilities — providing both usability and functionality.

1.3 EXISTING SYSTEM

Existing voice-based AI assistants such as **Google Assistant**, **Amazon Alexa**, **Apple Siri**, and **Microsoft Cortana** have set benchmarks in natural language interaction. These systems offer hands-free control, personalized assistance, and integration with various third-party services. However, they come with certain limitations:

1. **Limited Customization**: Most commercial assistants are tightly bound to their respective ecosystems. Users and developers have restricted access to underlying models, limiting customization and integration into third-party applications.

2. **Closed AI Models**: These assistants rely on proprietary AI models that do not allow developers to access or modify the conversational flow or knowledge base. This makes them unsuitable for educational, research, or domain-specific applications.

3. **Restricted Export and History Management**: Most existing systems do not provide flexible options for exporting chat history, especially in document formats like PDF or DOCX, which can be critical for professional or academic use.

4. **Lack of Open Integration**: Developers cannot easily integrate state-of-the-art models like **Gemini 1.5 Flash** or **OpenAI GPT** into their own systems without building a solution from scratch.

CHAPTER 2

2.1 PROPOSED SYSTEM

The proposed system, Jarvis, is a voice-activated AI assistant designed to overcome the limitations of existing commercial solutions. It integrates Google's Gemini 1.5 Flash LLM and OpenAI's GPT APIs within a lightweight Flask web framework to deliver a real-time, interactive, and intelligent assistant experience.

The system supports:

- Voice and Text Interaction: Users can either type or speak their queries.
- AI Response Generation: Queries are processed via Gemini or GPT models to produce natural language responses.
- Text-to-Speech Output: AI responses can be read aloud using gTTS, enhancing accessibility.
- Chat History Export: Users can download their interaction history in PDF or DOCX formats.
- Secure API Integration: Environment variables are used to protect API keys.
- Web Interface: Built using HTML, Tailwind CSS, and JavaScript for a modern, responsive user interface.

2.2 FLOW DIAGRAM

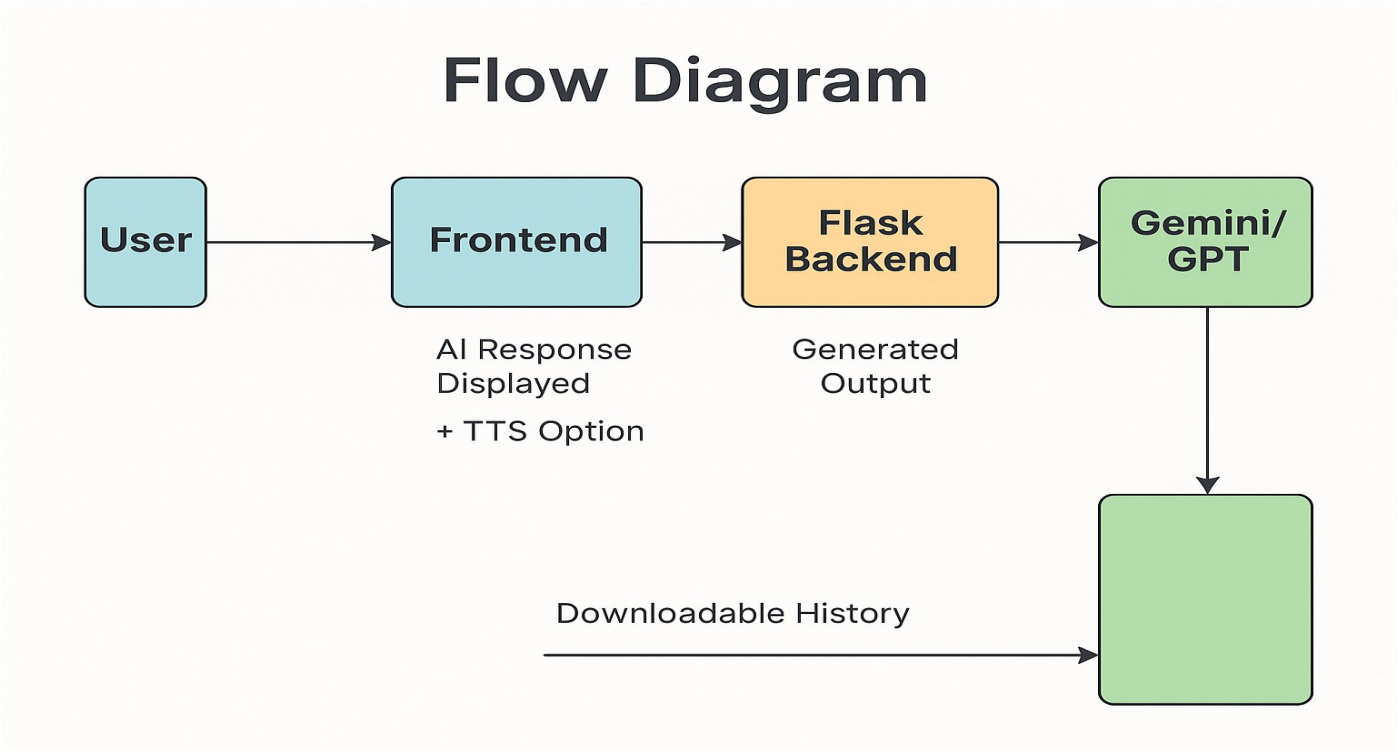

Fig 2.1: System Architecture Flow

- User Input: Through typing or speech.
- Frontend: Captures prompt and sends it to the backend.
- Flask Backend: Routes the request to Gemini/GPT, gets the response.
- LLM Processing: Gemini or GPT models generate the output.
- Frontend Display: Response is shown and can be spoken aloud or exported.

2.3 WORKING

1. Prompt Input: The user types or speaks a prompt using the browser interface.
2. Request Handling: The input is captured and sent to the Flask backend.
3. LLM Invocation: Based on the configuration, the backend forwards the prompt to Gemini or GPT API.
4. Response Processing: The LLM generates a coherent, contextual response.
5. Output Rendering: The frontend dynamically displays the AI-generated response in a styled chat interface.
6. TTS and Export: Users can click to hear the response or download the entire conversation log.
7. Speech Input Integration: A voice recognition feature allows users to input commands hands-free.

CHAPTER 3

IMPLEMENTATION

The implementation of the Jarvis AI assistant involves both frontend and backend development, integrated with advanced AI models and APIs. The system is built using Python's Flask framework and modern web technologies, making it lightweight, scalable, and user-friendly.

Frontend Implementation

- **Technologies Used**: HTML, JavaScript, Tailwind CSS
- **Interface Design**:
    - A single-page responsive UI
    - Input box for text prompts
    - Buttons for voice input (speech recognition), generate response, and download chat
    - Output container to display user and AI messages
- **Voice Input**: Integrated via the **Web Speech API** for real-time voice-to-text transcription
- **Text-to-Speech**: Integrated using **SpeechSynthesis API** for reading responses aloud
-

Backend Implementation

- **Technologies Used**: Flask (Python), gTTS, FPDF, python-docx
- **Endpoints**:
    - /generate: Receives user prompt and calls Gemini or GPT API to generate response
    - /generate-audio: Converts AI response to audio using gTTS
    - /download-pdf and /download-docx: Exports conversation history
- **LLM Integration**:
    - **Gemini 1.5 Flash** is configured using the google.generativeai package
    - **OpenAI GPT** is accessed through openai package
    - Responses are timestamped and stored in a chat history list for export

Security

- API keys for Gemini and OpenAI are stored in a .env file using python-dotenv, preventing exposure in source code.
- Flask handles all HTTP requests and securely communicates with AI models.

Deployment

- Deployed using **Render** for live hosting, offering:
    - Auto-deployment from GitHub
    - SSL (HTTPS) by default
    - Scalable infrastructure

CHAPTER 4

SOURCE CODE

4.1 FrontEnd

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Jarvis</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <style>
    body {
      font-family: 'Inter', sans-serif;
    }


    .output-entry-prompt {
      margin-bottom: 15px;
```

```
    padding-bottom: 15px;
    border-bottom: 1px dashed #cbd5e1;
  }

  .user-line, .bot-line {
    display: flex;
    align-items: center;
    gap: 10px;
    margin-bottom: 8px;
  }

  img {
    max-width: 26px;
    max-height: 22px;
  }

  #loading {
    display: none;
    margin: 10px 0;
    font-style: italic;
  }
 </style>
</head>

<body class="overflow-x-hidden bg-[#131415] text-white">
 <div class="flex flex-col lg:flex-row min-h-screen">

  <!-- Sidebar -->
   <div class="w-full lg:w-48 bg-[#1e293b] p-4 flex lg:flex-col flex-row gap-2 lg:gap-4 justify-around
items-center shadow-md sticky top-0 z-10">
```

```
    <h2 class="text-xl font-semibold mb-2 lg:mb-4">Downloads</h2>


      <button onclick="download('pdf')" class="bg-green-500 hover:bg-green-600 px-4 py-2 rounded text-white w-full lg:w-auto">⬇ History PDF</button>

    <button onclick="download('docx')" class="bg-yellow-500 hover:bg-yellow-600 px-4 py-2 rounded text-white w-full lg:w-auto">⬇ History DOCX</button>

  </div>


  <!-- Main -->

  <div class="flex-1 p-4 overflow-y-auto">

      <div class="max-w-4xl mx-auto bg-gradient-to-br from-blue-900 to-slate-800 p-6 rounded-xl shadow-xl">

      <h1 class="text-3xl font-bold text-center mb-6">Jarvis</h1>


      <div id="output-container" class="overflow-y-auto max-h-[45vh] bg-white text-black p-4 rounded mb-4">

        <p class="text-center">Your generated text will appear here.</p>

      </div>


      <div id="loading" class="text-center text-white">Generating...</div>


      <input type="text" id="input" placeholder="Enter your prompt here..."

        class="w-full p-3 rounded bg-slate-100 text-black text-lg outline-none" />


      <div class="flex flex-wrap justify-center mt-4 gap-3">

        <button id="generateButton" class="bg-indigo-500 hover:bg-indigo-600 px-4 py-2 rounded text-white">Generate</button>

        <button onclick="startListening()" class="bg-red-500 hover:bg-red-600 px-4 py-2 rounded text-white">□ Speak</button>

      </div>

    </div>
```

```
  </div>
</div>

<script>
  const inputField = document.getElementById('input');
  const generateButton = document.getElementById('generateButton');
  const outputContainer = document.getElementById('output-container');
  const loadingDiv = document.getElementById('loading');

  generateButton.addEventListener('click', async () => {
    const promptText = inputField.value.trim();
    if (!promptText) return;

    loadingDiv.style.display = 'block';
    generateButton.disabled = true;

    try {
      const res = await fetch('/generate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ text: promptText })
      });
      const data = await res.json();

      if (outputContainer.querySelector('p')) {
        outputContainer.querySelector('p').remove();
      }

      const div = document.createElement('div');
```

```
      div.classList.add('output-entry-prompt');


      const escapedPrompt = promptText.replace(/"/g, '&quot;').replace(/'/g, '&#39;');
      const escapedResponse = data.generated_text.replace(/"/g, '&quot;').replace(/'/g, '&#39;');


      div.innerHTML = `
        <div class="user-line">
          <img src="/static/userr.png" alt="User" />
          ${promptText}
        </div>
          <button onclick="speakOut(this)" data-text="${escapedPrompt}" class="text-sm text-blue-700 hover:underline mt-1 mb-2">🔊</button>
        <div class="bot-line" style="justify-content: flex-end;">
          <div style="text-align: justify;">${data.generated_text}</div>
          <img src="/static/bot.png" alt="Bot" />
        </div>
          <button onclick="speakOut(this)" data-text="${escapedResponse}" class="text-sm text-blue-700 hover:underline">🔊</button>
        <div class="text-xs text-right text-gray-600 mt-2">${data.timestamp}</div>
      `;
      outputContainer.appendChild(div);
      inputField.value = '';
    } catch (error) {
      console.error('Error:', error);
    } finally {
      loadingDiv.style.display = 'none';
      generateButton.disabled = false;
    }
  });
```

```
inputField.addEventListener('keypress', (e) => {
  if (e.key === 'Enter') generateButton.click();
});

function speakOut(el) {
  const text = el.dataset.text || el;
  const utterance = new SpeechSynthesisUtterance(text);
  speechSynthesis.cancel();
  speechSynthesis.speak(utterance);
}

async function generateAudio() {
  const prompt = inputField.value;
  const res = await fetch('/generate-audio', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ text: prompt })
  });
  const blob = await res.blob();
  const audioUrl = URL.createObjectURL(blob);
  const audio = new Audio(audioUrl);
  audio.play();
}

function download(type) {
  const url = type === 'pdf' ? '/download-pdf' : '/download-docx';
  window.location.href = url;
}
```

```
    function startListening() {
      const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
      recognition.lang = 'en-US';
      recognition.interimResults = false;
      recognition.maxAlternatives = 1;

      recognition.onresult = (event) => {
        inputField.value = event.results[0][0].transcript;
      };

      recognition.onerror = (event) => {
        console.error("Speech recognition error", event.error);
      };

      recognition.start();
    }
  </script>
</body>
</html>
```

4.2 BackEnd

```
import os
from flask import Flask, request, jsonify, render_template, send_file
import google.generativeai as genai
from gtts import gTTS
from io import BytesIO
from fpdf import FPDF
from docx import Document
from dotenv import load_dotenv
```

```
from openai import OpenAI
from datetime import datetime

load_dotenv()
GOOGLE_API_KEY = os.getenv("GOOGLE_GENAI_API_KEY")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

if not GOOGLE_API_KEY or not OPENAI_API_KEY:
    raise ValueError("API keys are not set in environment variables")

genai.configure(api_key=GOOGLE_API_KEY)
model = genai.GenerativeModel('gemini-1.5-flash')
client = OpenAI(api_key=OPENAI_API_KEY)

tmpl_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'templates')
app = Flask(__name__, template_folder=tmpl_dir, static_folder="static")

chat_history = []

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/generate', methods=['POST'])
def generate():
    try:
        text = request.json['text']
        response = model.generate_content(text)
        generated_text = response.text
```

```
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        chat_history.append({'prompt': text, 'response': generated_text, 'timestamp': timestamp})
        return jsonify({'generated_text': generated_text, 'timestamp': timestamp})
    except Exception as e:
        return jsonify({'error': str(e)}), 500


@app.route('/generate-audio', methods=['POST'])
def generate_audio():
    try:
        text = request.json['text']
        tts = gTTS(text)
        audio_data = BytesIO()
        tts.write_to_fp(audio_data)
        audio_data.seek(0)
        return send_file(audio_data, mimetype='audio/mpeg', as_attachment=False)
    except Exception as e:
        return jsonify({'error': str(e)}), 500
@app.route('/download-pdf')
def download_pdf():
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial", size=12)
    for entry in chat_history:
        pdf.multi_cell(0, 10, f"User: {entry['prompt']}\nJarvis: {entry['response']}\nTime: {entry['timestamp']}\n")
    pdf_path = "chat_history.pdf"
    pdf.output(pdf_path)
    return send_file(pdf_path, as_attachment=True)
@app.route('/download-docx')
def download_docx():
```

```
    doc = Document()
    doc.add_heading('Chat History', 0)
    for entry in chat_history:
        doc.add_paragraph(f"User: {entry['prompt']}")
        doc.add_paragraph(f"Jarvis: {entry['response']}")
        doc.add_paragraph(f"Time: {entry['timestamp']}")
        doc.add_paragraph("")
    doc_path = "chat_history.docx"
    doc.save(doc_path)
    return send_file(doc_path, as_attachment=True)
if __name__ == '__main__':
    app.run(debug=True)
```

CHAPTER 5

ADVANTAGES AND DISADVANTAGES

5.1 Advantages

1. **Real-Time AI Interaction**
   The system provides real-time responses using powerful LLMs (Gemini & GPT), enabling intelligent, human-like conversations.
2. **Multimodal Input & Output**
   Supports both voice and text inputs. The system also offers text-to-speech playback, enhancing accessibility for all users.
3. **Lightweight Web Architecture**
   Built using Flask and Tailwind CSS, the application remains lightweight, fast, and easy to deploy or scale.
4. **Secure API Integration**
   API keys are safely stored using .env files, avoiding exposure and protecting sensitive credentials.
5. **Downloadable Chat History**
   Users can export their conversations as **PDF** or **DOCX**, making the tool useful for documentation and learning purposes.
6. **Open and Extensible**
   The modular architecture allows developers to extend or customize features easily, such as authentication, multi-language support, or database integration.
7. **Deployed on the Cloud**
   Hosted on **Render**, enabling public access, continuous deployment, and real-world usage.

5.2 Disadvantages

1. **Limited Offline Functionality**
   The system depends on internet access and API availability. Offline usage is not supported.

2. **Dependency on External APIs**
   Gemini and GPT APIs are third-party services. Any API limitations or pricing changes can affect functionality.

3. **No Persistent Chat Storage**
   Current implementation stores chat history only during runtime. Restarting the server clears all previous conversations.

4. **Limited Multilingual Support**
   The current version primarily supports English. Additional languages would require further integration and testing.

5. **Security Enhancements Needed**
   Authentication, authorization, and encryption mechanisms are not yet implemented and are necessary for production-level deployment.

CHAPTER 6

RESULTS

1. Screen Shot 1

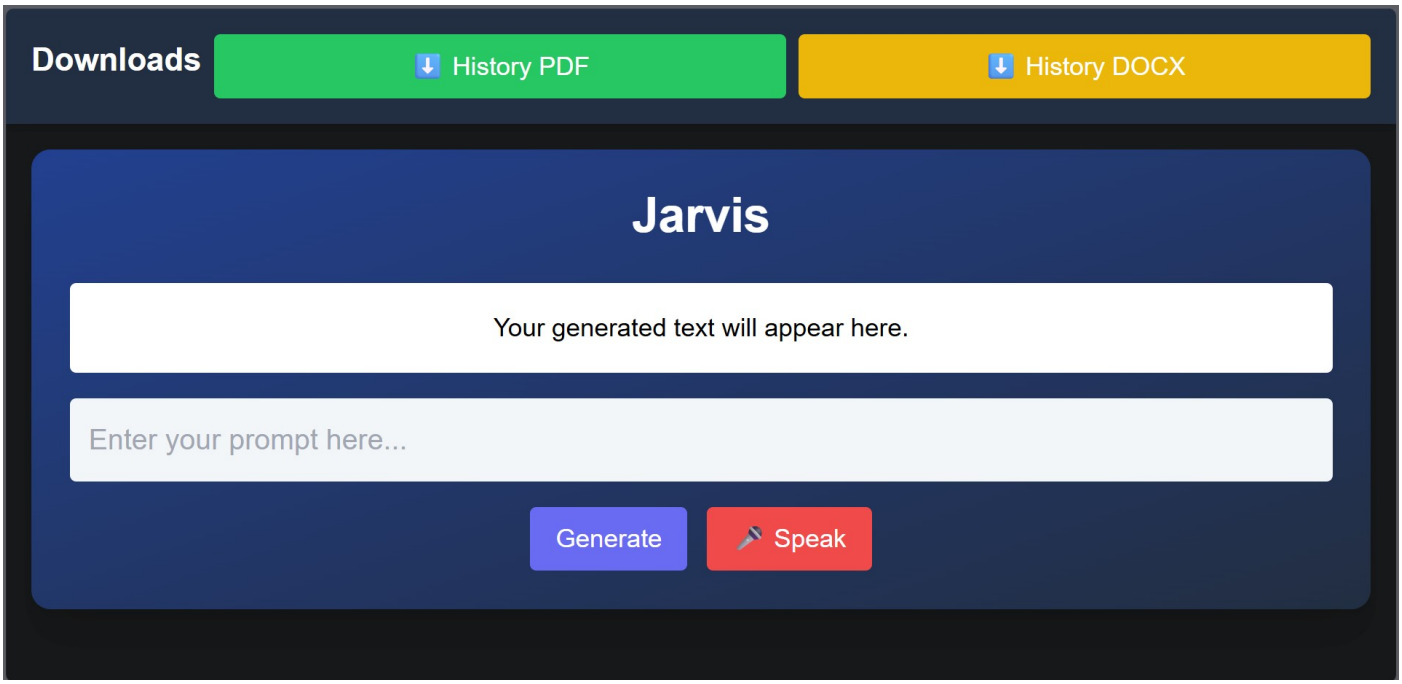

Fig 6.1: Home Page

2. Screenshot 2

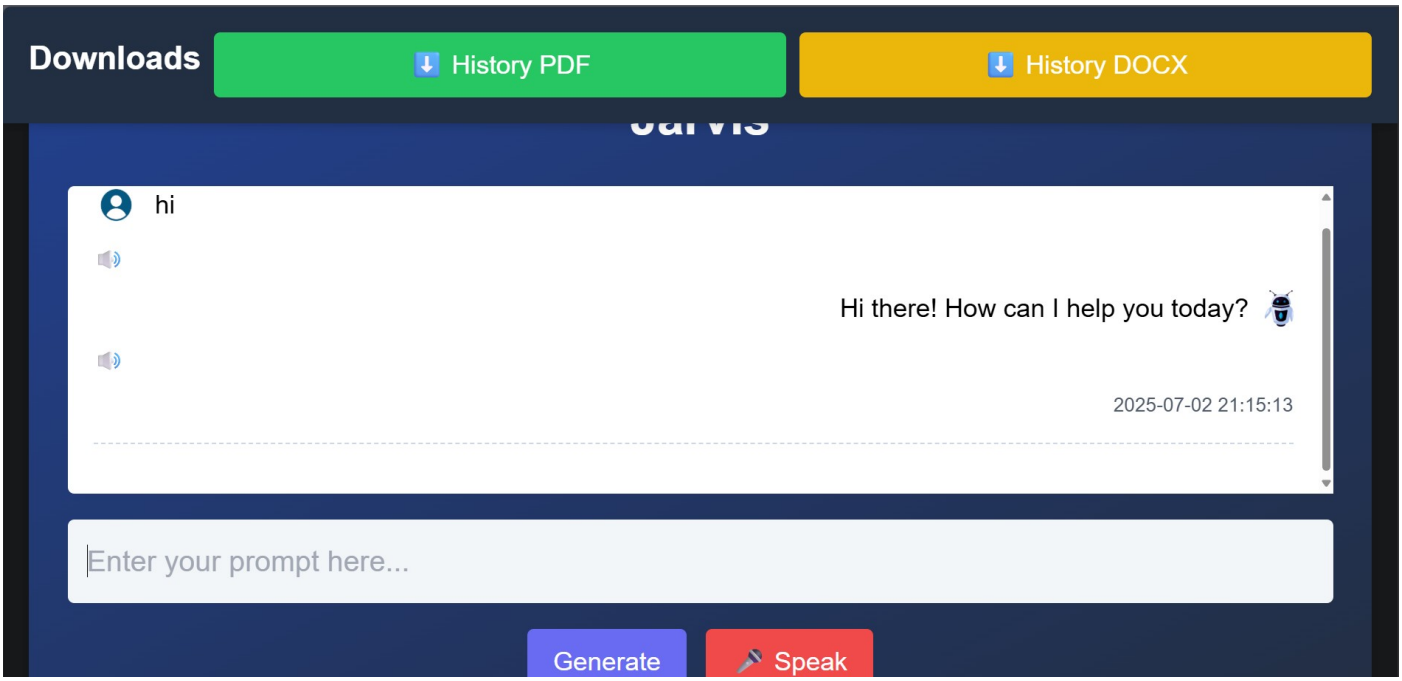

Fig 6.2: Working of Model

3. Screenshot 3

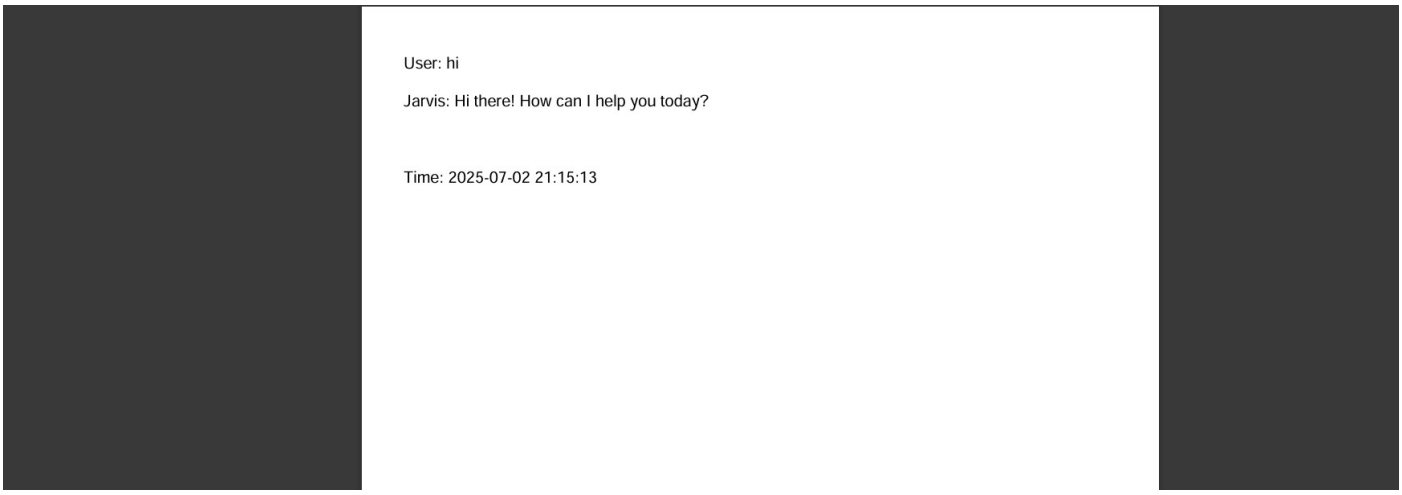

Fig 6.3: Download of PDF

4. Screenshot 4

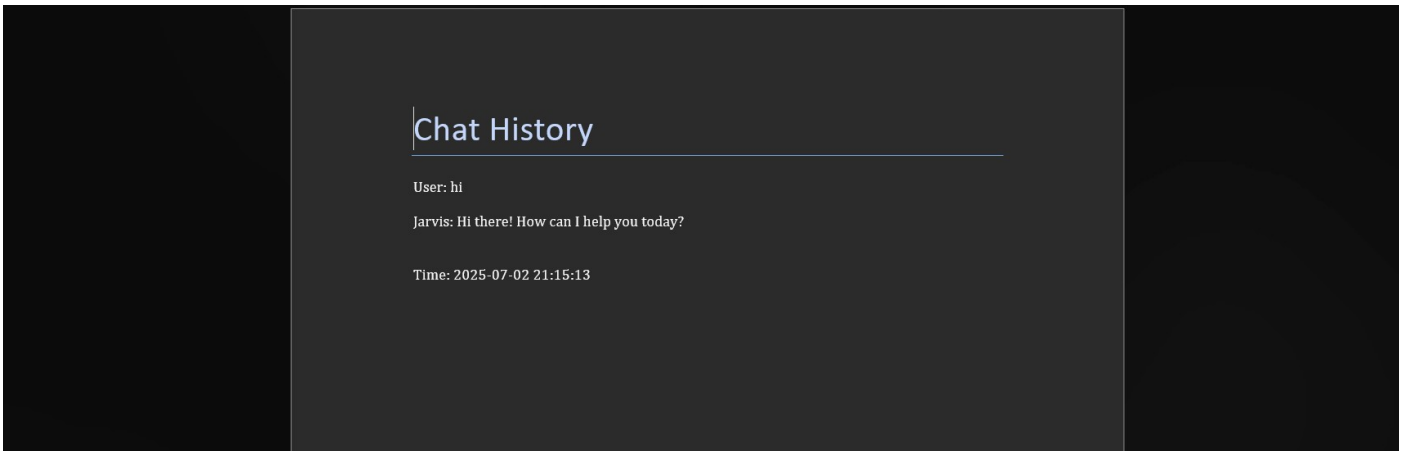

Fig 6.4: Download of DOCX

CHAPTER 7

CONCLUSION

The system developed in this project stands as a fully operational prototype that showcases the successful integration of **Google's Gemini 1.5 Flash Large Language Model** and **OpenAI's GPT** API with a modern **Flask-based web application**. Through this integration, users are able to interact in real-time with the AI via a clean and responsive user interface. The application supports both **textual and voice input**, delivers AI-generated responses, and enhances user experience with features like **text-to-speech**, **speech recognition**, and **downloadable conversation history** in **PDF** and **DOCX** formats.

This implementation not only demonstrates the technical feasibility of embedding LLMs into modern web platforms but also sets a strong foundation for intelligent digital assistants, educational platforms, and

interactive AI-based tools. While the current version highlights core functionality and real-time communication, its modular design allows for **easy extensibility**. With continued development—such as the inclusion of authentication, persistent data storage, and scalable deployment—this project has the potential to evolve into a production-ready solution capable of serving users across domains such as **customer service**, **tutoring**, **content generation**, and **personal productivity**.

CHAPTER 8

REFERENCES

1. Google, “Google Generative AI – Gemini 1.5 Flash,” Google AI Blog, 2024. [Online]. Available: https://deepmind.google/technologies/gemini/
2. OpenAI, “OpenAI GPT API Documentation,” OpenAI, 2024. [Online]. Available: https://platform.openai.com/docs
3. Flask, “Flask: Web Development, One Drop at a Time,” Pallets Projects, 2024. [Online]. Available: https://flask.palletsprojects.com/
4. Tailwind Labs, “Tailwind CSS: Rapidly build modern websites without ever leaving your HTML,” 2024. [Online]. Available: https://tailwindcss.com
5. Google, “gTTS: Google Text-to-Speech,” PyPI, 2024. [Online]. Available: https://pypi.org/project/gTTS/

6. PyFPDF, “FPDF: A Python class to create PDF files,” 2024. [Online]. Available: https://pyfpdf.github.io/fpdf2/
7. Python Software Foundation, “python-docx: Create and update Microsoft Word .docx files,” PyPI, 2024. [Online]. Available: https://pypi.org/project/python-docx/
8. Mozilla Developer Network, “Web Speech API – Speech Recognition,” MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
9. Render, “Render: Cloud Application Hosting for Developers,” 2024. [Online]. Available: https://render.com/
10. dotenv-labs, “python-dotenv: Reads key-value pairs from a .env file and adds them to environment variables,” GitHub, 2024. [Online]. Available: https://github.com/theskumar/python-dotenv